INDIAN INSTITUTE OF TECHNOLOGY MADRAS

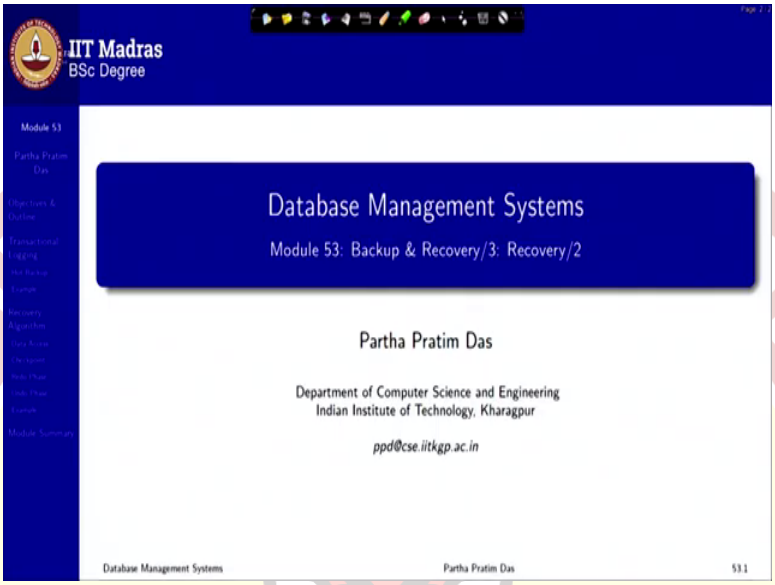
सिद्धिर्भवति कर्मजा

IIT Madras

ONLINE DEGREE

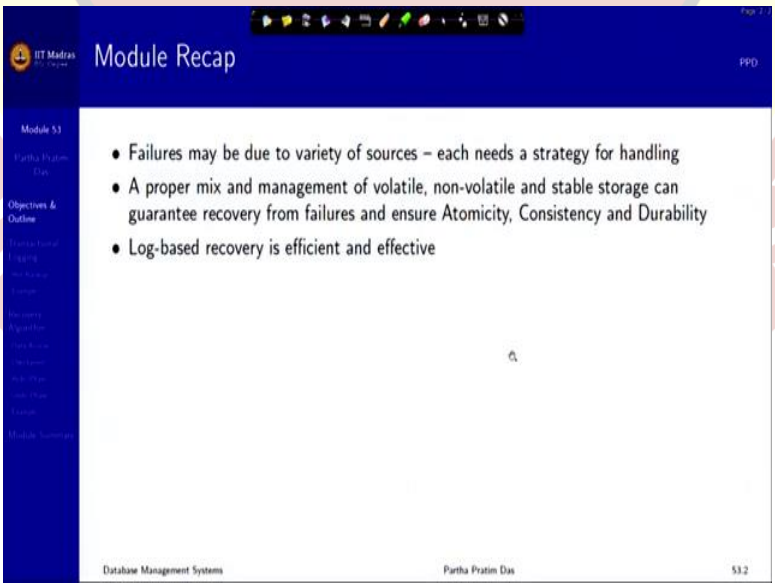
Database Management Systems
Professor Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology Kharagpur
Backup & Recovery 3: Recovery 2

(Refer Slide Time: 0:24)



Welcome to module 53 of Database Management Systems course in the IIT Madras online B.Sc. program.

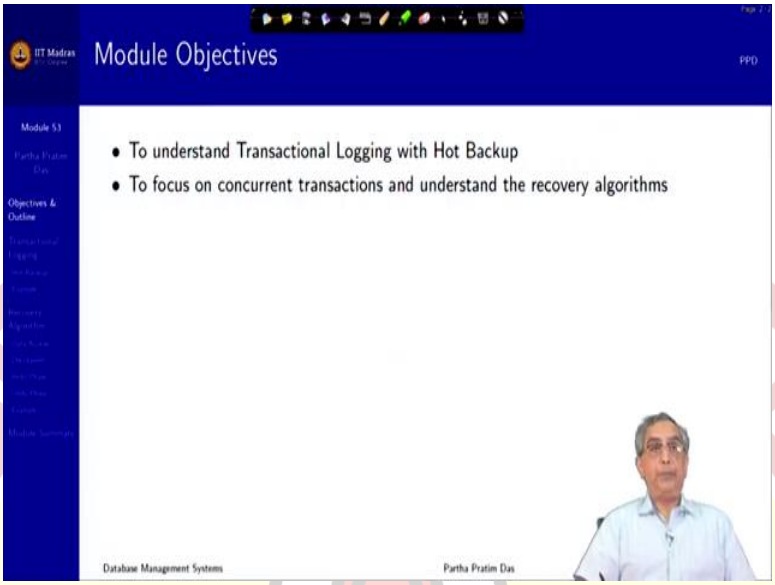
(Refer Slide Time: 0:33)



We have been discussing about backup and recovery the key factors to ensure atomicity, consistency and durability because there can be failure due to variety of sources as we have

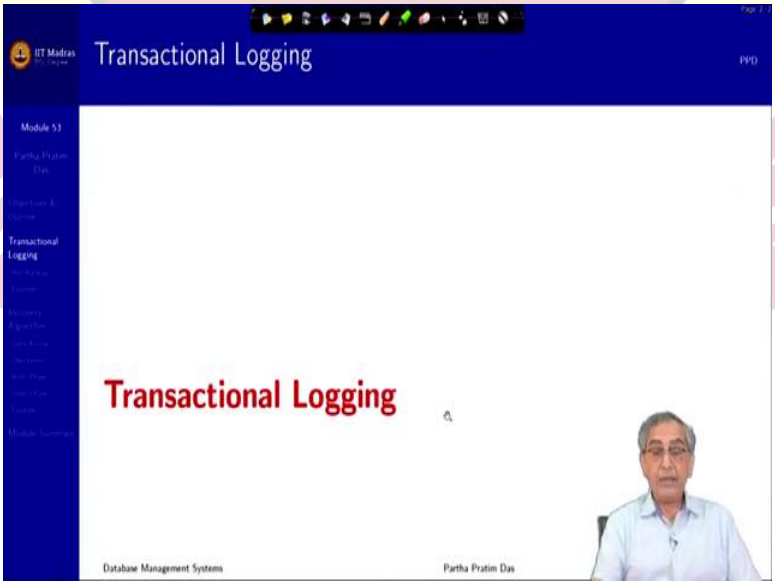
seen. But with a proper mix of volatile, nonvolatile as well as stable storage we can guarantee a perfect recovery and log-based recovery as we have seen is efficient and effective.

(Refer Slide Time: 1:00)



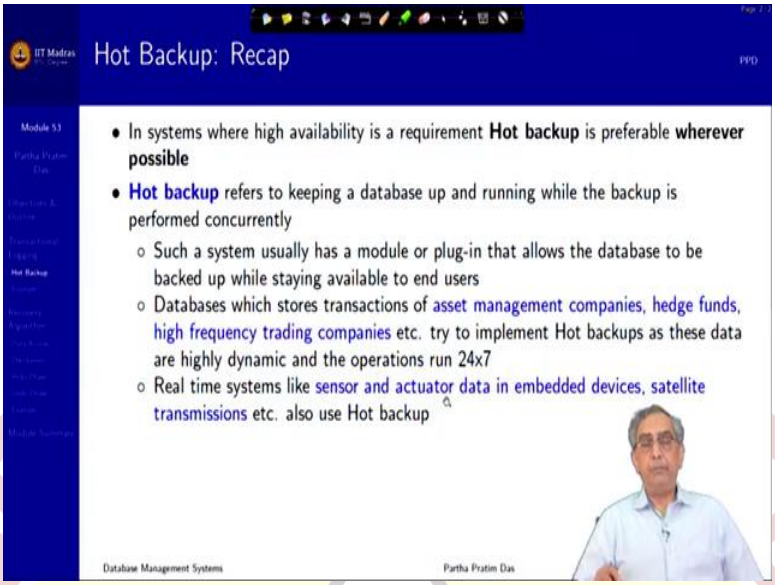
In a module today we will try to understand transactional logging with hot backup in some more depth and then focus on concurrent transactions and their recovery. We had in the last module discussed about serial transactions and their recovery, so we will see how that will generalize to concurrent transactions, so that is the outline.

(Refer Slide Time: 1:22)



So, transactional logging.

(Refer Slide Time: 1:25)

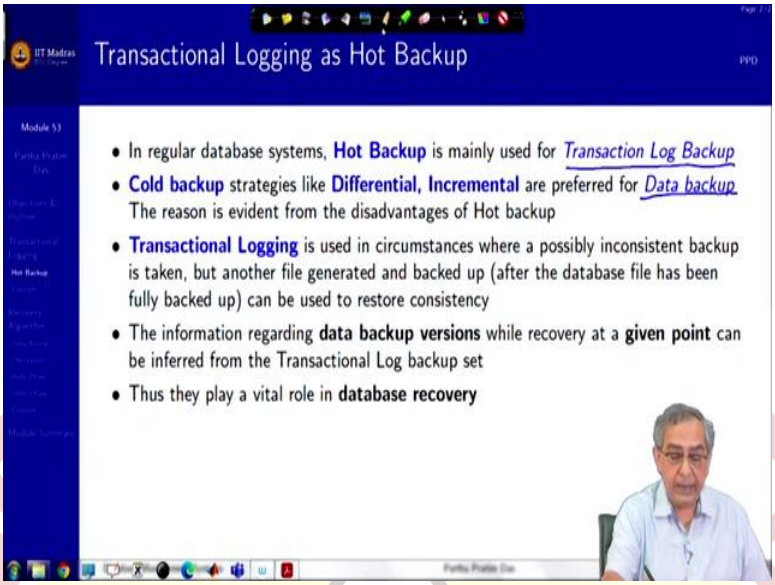


Let us understand the basic issue. See we have what we have identified very correctly is a backup is a necessity to preserve the correctness of the data, consistency of the database. And we have seen different forms of cold backup at different granularity of time in terms of full differential incremental and so on.

Now, the question is if a database would fail in terms of system failure, power failure, disk failure, while it is executing the transaction how do you recover from that point? So, that requires a hot backup to be done, that is as and when you are doing the transaction you will also make a backup of that in a high speed possibly a nonvolatile RAM and things like that which does not take much time, does not need to write much data but can be fully recovered.

So, this is very critical for all database applications which has stores transactions like asset management, hedge funds, stock trading, different kinds of banking applications and so on even in real time applications, we will need this very frequently.

(Refer Slide Time: 2:55)

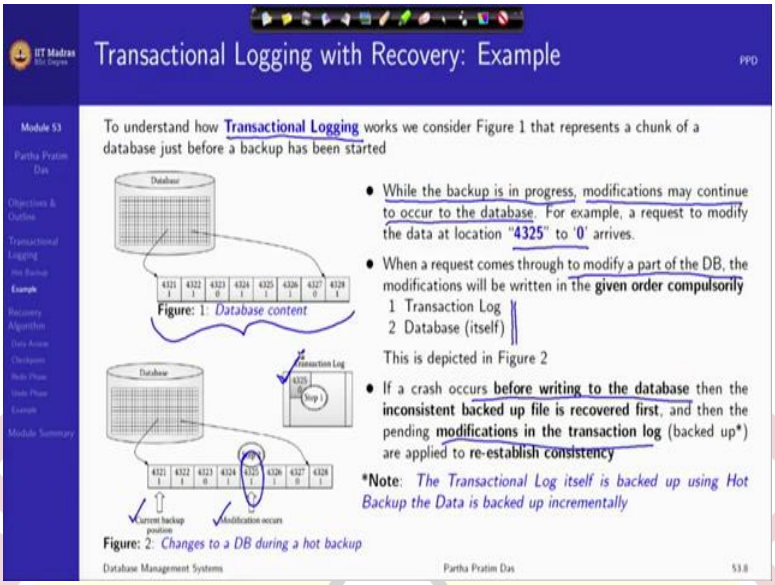


Now, the point is if we have to have hot backup for the database that is for the B plus tree files that we are having as a database, then it will become extensively expensive to have a hot backup storage. And it will become almost infeasible to do that. So, the logging has been introduced as a concept to work around this.

So, what do we say that we log every action that a transaction does and as we will see that it is not only limited to what operations read writes transactions are doing but we will also extend this to actual databases own application updates. Like in index update with insert or say update of free space tables and so on.

So, what we use hot backup for is primarily the backup of transaction logs because they are much smaller in size, whereas the cold backup is used like the full differential incremental and all that for actual data backup. So, whenever there is a failure from the last cold backup point up to which we can obviously fully restore the data backup we will be using the transaction log backup from the hot backup to update the database from that point to the point where the failure, the crash actually happens and that is very critical, plays a pivotal role in the database recovery.

(Refer Slide Time: 4:48)



Now, let us take an example, a sample scenarios to understand how transaction logging actually works and solves the problem of inconsistency in the database due to unfortunate crash and so on using the hot backup of the transaction log. So, let us look at figure 1, so what it shows is just a chunk of data from the database, I mean this is just representative.

And this chunk is shown just before a backup has been started, so at this point this is a from 4321 to 4378 these are the data for simplicity we are just keeping the data as 1 or 0, they are just placeholders, it, in reality they could be more complex data. But this is the snapshot when a backup starts.

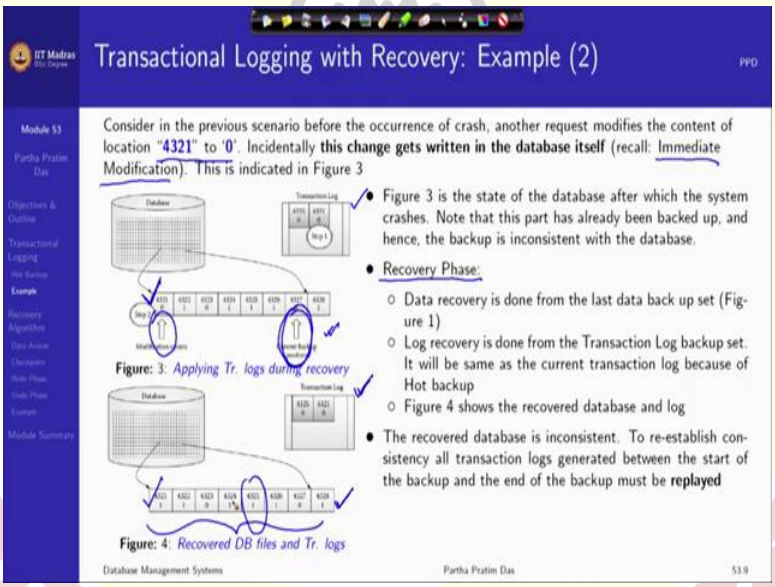
Now, while the backup is in progress so what would be right to ensure consistency? One could be while we are doing this backup, we stop the operations in the database, just hold everything, do the backup, finish it, if it is okay, then we start it again, that will certainly be a simple solution but that is not something which is affordable because most of these applications are 24 by 7, so you cannot stop the operation while the backup is going on. So, you will have to continue to let the database operate so modifications may continue to occur in the database, so that is the basic.

Now, suppose for example the location 4325 should be changed to 0 that is the write request that you get. Now, comes the critical point as to how because you can see we are in this say second figure if you see then this is the backup has started at this point so you are planning to make changes to 4325 which is in the scope of the backup but has not been backed up yet.

Now, if something will go wrong during this time how do you, how will that be taken care of? So, the protocol would be very simple when a request comes to modify a part of the database, the modifications will be written in this order. That is first you will write it to the transaction log and mind you the transaction log is in a hot backup. So, if something goes wrong even then the latest status of the transaction log would be available.

And then later you write into the database. So, if the crash occurs before writing to the database, this is the step 2, you have not been able to write it yet there but then if the crash occurs at that time then the inconsistent backup file will be recovered first and then the modified transaction log which remembers that 4325 was to be changed to 0 will be applied to re-establish that consistency, so that is the basic principle of the process. So, let us continue and see what other scenarios can happen.

(Refer Slide Time: 8:22)



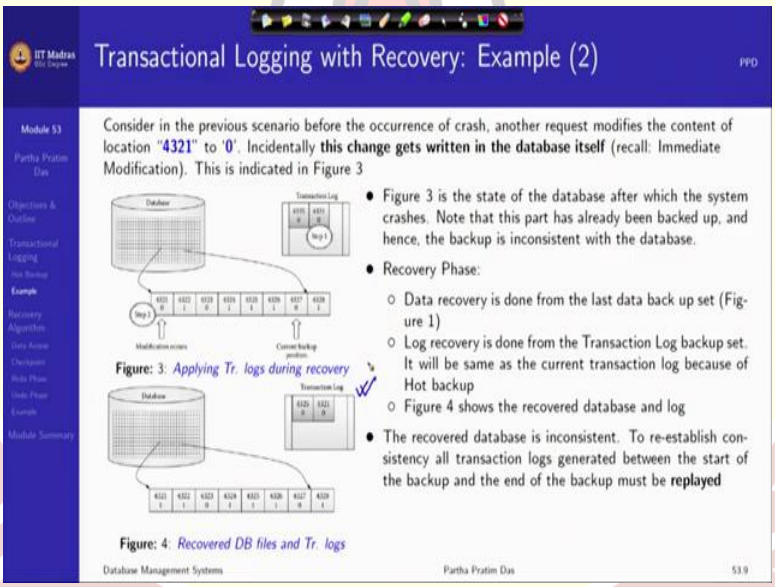
Now, let us again go back to the previous scenario before the occurrence of the crash, the crash has not happened yet let us say. And this is the status, this is your modification status, this is where what you are trying to do in this is you are trying to write 0 to 4321 and let us assume that we are using a immediate modification scheme for this.

So, it will be written to the database immediately, of course following the protocol that the first transaction log has to be written and then it is written in the database. Mind you your current backup position is here, so you are writing to a point where the backup has already happened. So, backup has a different value one, and you have changed it after that because of the immediate modification at 4321 and this is so certainly the database and the backup are not consistent at this moment and backup has not ended either.

Now, with this so if a crash will happen at this point what will happen? So, you will have to recover from the backup. When you recover from the backup since you had not finished this backup when you recover from that and you have made changes to the database also, your recovery would be back to the original point where you had started the backup, so that is figure 1, which is the state we started with. Because the backup got interrupted because of some crash.

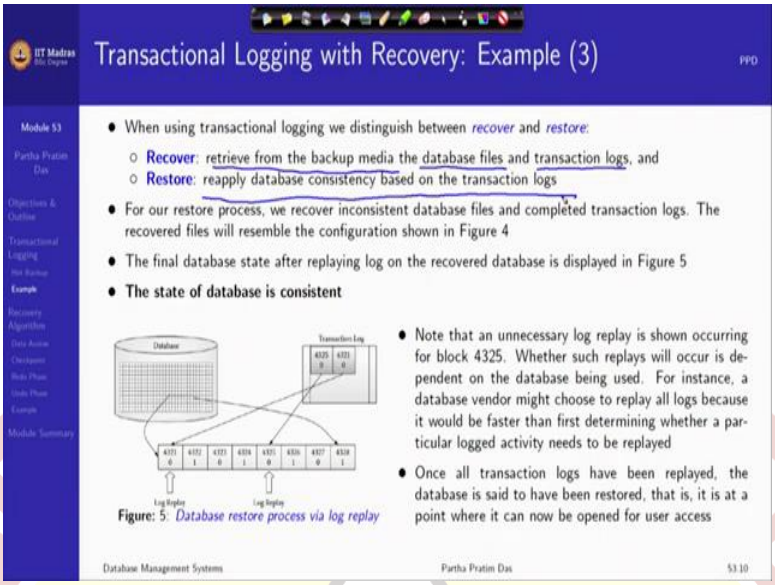
Now, the log will also have to be recovered and that recovery is being assumed to be hot backup so the log will be totally nothing will be lost in the log, so it will be same as the current transaction log. So, you had written this here, you get that information. So, this is what you have got and you can see that the change that you had made here due to immediate modification in the database has now been lost but according to your notion the change has been made and the change that was to be made in 4325 obviously has not been done yet.

(Refer Slide Time: 11:01)



So, after this backup you get into an inconsistency so to reestablish the consistency you have to replay the transaction log, you have to replay this transaction log. So, let us go to the next.

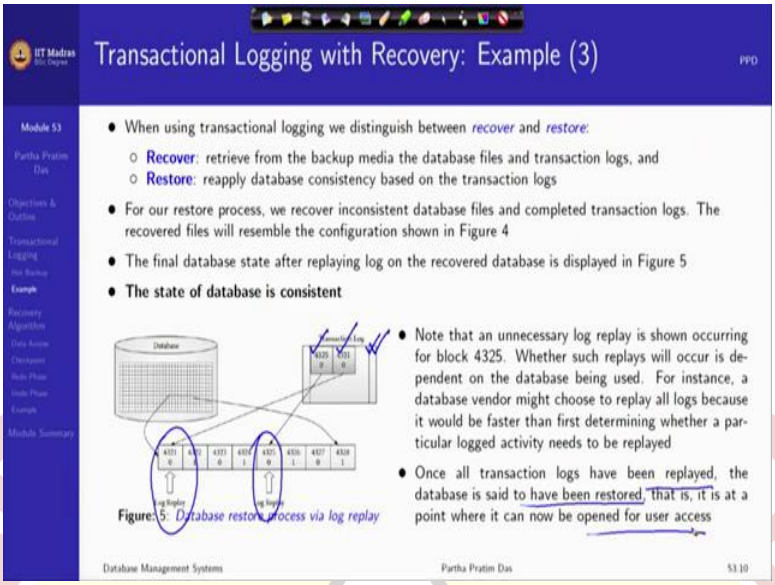
(Refer Slide Time: 11:08)



So, in this process we will specifically use two words, one is we say is recovery or recover, other we say is restoration or restore. The recovery means to recover means you retrieve the backup from the backup media wherever you have done a backup, it could be a tape or some other disk whatever.

And retrieve the database files that was last correctly backed up and the transaction log that is the recovered process. And after you have recovered potentially your database and the database is in a inconsistent state because your backup and database had become inconsistent so you reapply the database consistency based on the transaction log that is called the restore process. So, we have already done the recover as in the previous slide.

(Refer Slide Time: 12:05)

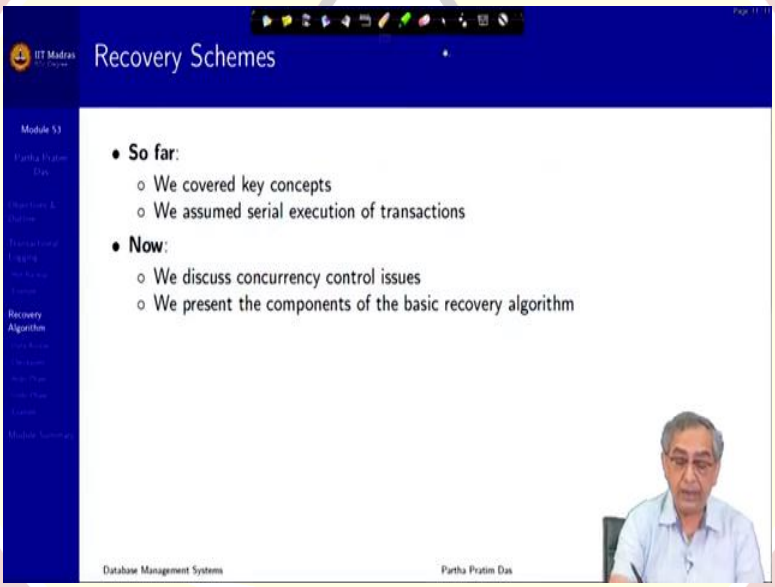


So, to restore we will have to replay this transaction log. So, the transaction log says that 4325 should become 0, this, you replay this log and make it 0, 4321 should be 0 so you replay this log and make it 0. And once you have done this restoration your database is again in a consistent state. So, this is the basic process of restore and, recover and restore that can be facilitated when you have hot backup on the transaction logging to make sure that whenever you have a crash it will not lead to a database inconsistency, you will always be able to recover from the last backup and restore using the transaction log.

Now, while you do that obviously the questions will be as to well in the after the recovery some of the data which you would replay according to the transaction log may already be correct. So, would you try to find those and apply transaction log selectively, usually that is not what is done, usually vendors do not do that because it is more complex to find out what has been updated and what has not been, it is safer may be little bit of extra work, but it is safer to replay the entire transaction log, because you know that the transaction log corresponds to the last state of the backup, and therefore replaying it will always bring it to the current consistent state.

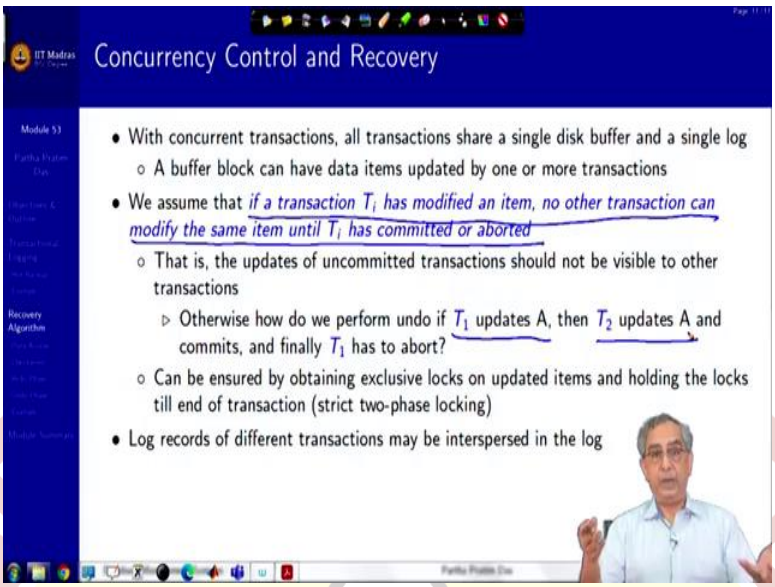
And once all transaction logs have been replayed the database has been restored and then it can be opened for user access. And please remember that the transaction log will keep on building up like this. So, periodically the transaction log will have to be updated on the database and you have to start on a fresh log which is what the vendor decides on the time period of doing that based on the nature of the application and the type of the database system.

(Refer Slide Time: 14:16)



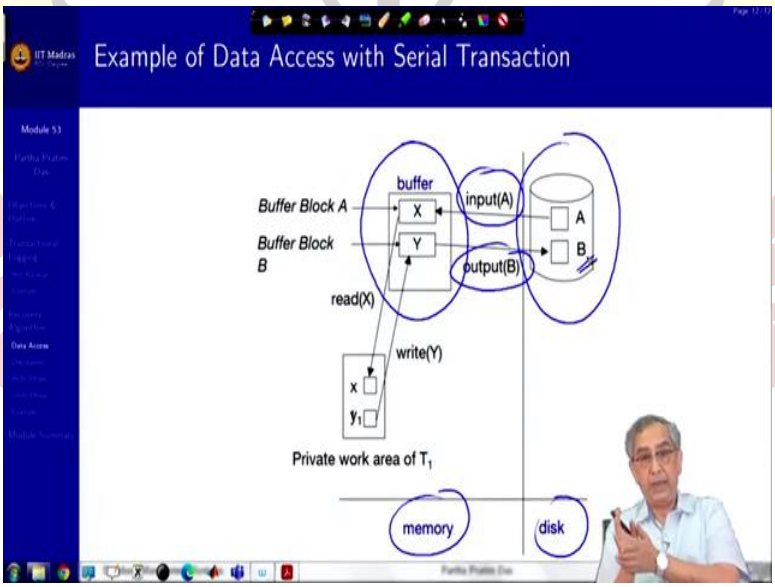
And in this context we had seen the recovery strategy for serial execution of transactions, now we will generalize this for concurrent control issues, the concurrent execution of transactions.

(Refer Slide Time: 14:31)



So, in this we make an assumption that if a transaction Ti has modified an item then no other transaction can modify the same item either until Ti has committed or aborted, this is the restriction that we maintain otherwise when we perform undo for Ti's update of A and T2 has also updated A then since Ti has not committed or aborted, the rollback will not be possible to be done in a consistent way.

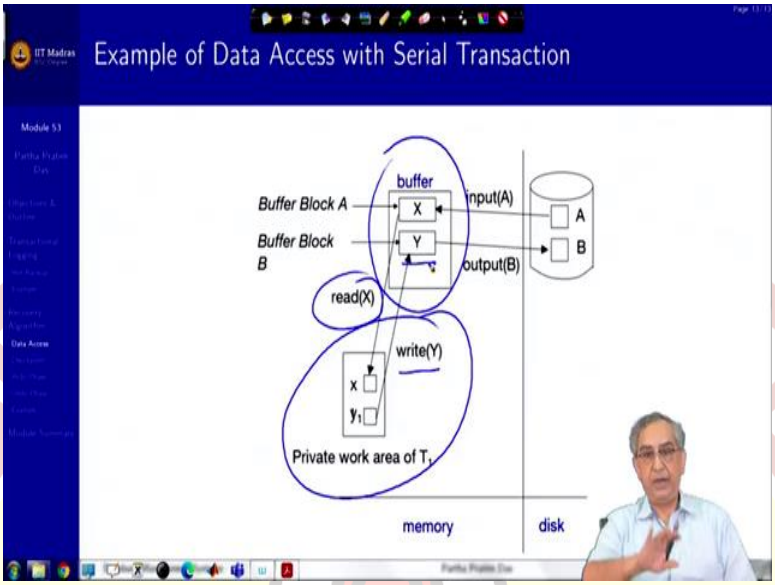
(Refer Slide Time: 15:15)



Given that this is the basic model that we had shown for serial transactions that this is your persistent data that is your disk data from where you bring to memory buffer this is in disk, this is part is in the memory. And you bring the variable A into buffer say X, this operation is

input A which is reading the disk from the disk A block. Similarly, you can write back a value Y into B which is writing, back the block back to the disk.

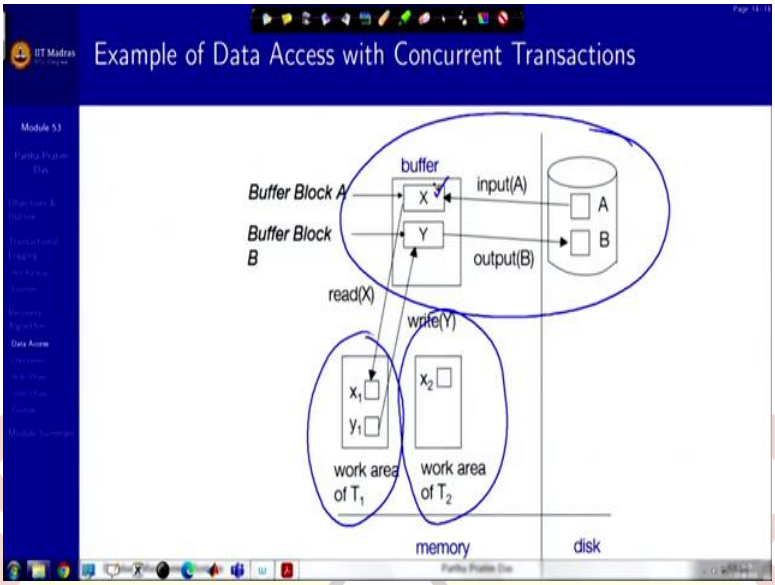
(Refer Slide Time: 15:54)



And given this buffer the transaction say a transaction T1 actually does not directly operate on the buffer but it has its own private buffer where it is making the changes, it reads from the buffer for the first time if it needs to, that is read X and when the, when a value is ready it will write and write to this buffer.

So, when it writes then it may happen that the output will happen immediately or the output may wait, the output may happen before the transaction T1 commits, it may happen after transaction T1 commits all of this kind of possibilities are there but since these two are dissociated, the read write is dissociated from input output, you will not get into any difficulty in handling this.

(Refer Slide Time: 16:50)

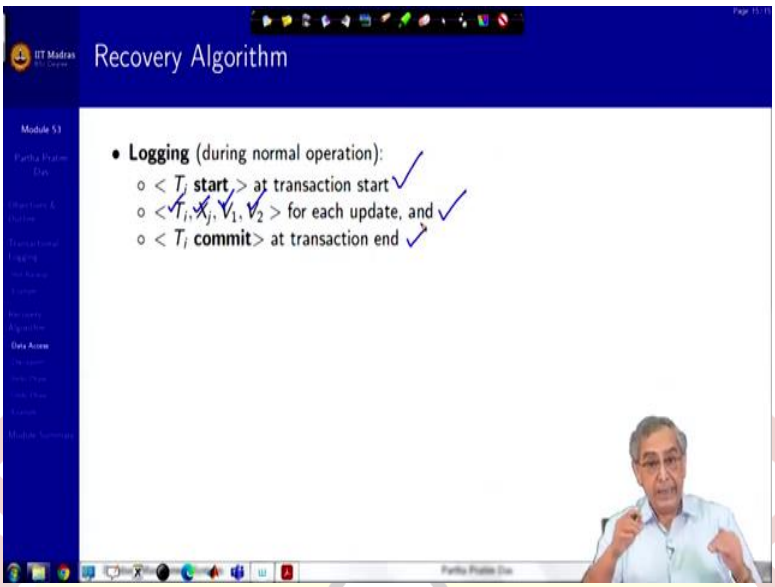


Now, when we go to concurrent transaction naturally this part remains the same that you are still reading from the disk by block and the memory buffer block may have the data of multiple transactions in the same buffer. But every transaction has its own separate buffer area, this is not common.

So, whatever T1 does in its buffer area will not be seen or editable in any way by T2 and same is whatever T2 does in its own work area, buffer area, transaction buffer area T1 will not be able to. So, they kind of so as many transactions as you have you will have separate buffer areas if you are familiar with the concept of threads it is very similar to that, every thread has its own stack, so it is a similar kind of concept like that.

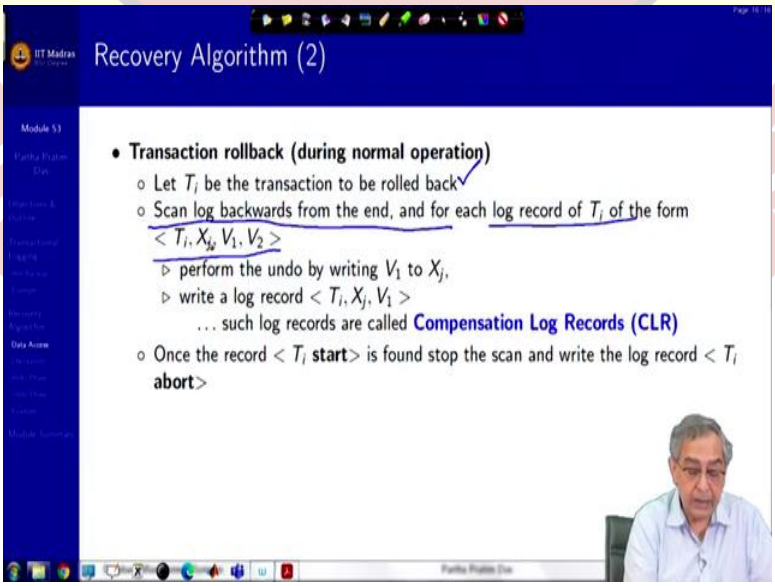
And whenever it needs the data whether T1 or T2 it reads from the buffer for the first time, the input buffer from the first time and keeps it there. And whenever it is finally ready with a value it writes it back. So, what happens is you can now see that since multiple transactions are writing into the buffer block it is the buffer management will have to decide as to when it outputs it. If every time a transaction writes from its own buffer to the memory buffer block and you have to output to the disk, then naturally it will be very slow. So, it will decide based on what is an appropriate time for doing the actual write.

(Refer Slide Time: 18:25)



So, the recovery algorithm as we saw during the normal operation would have a logging which is very simple in terms of T1, every transaction has a log T1, Ti start. It has for each update that it does of a value change, it will have Ti changing the value for updating the value for Xj from V1 this is the old value to V2 which is a new value, it will keep on writing all that in the log so you will know what changes have happened and at the end the transaction commits and writes this log. So, these are the log entries available for the recovery algorithm during a normal operation of a transaction.

(Refer Slide Time: 19:17)

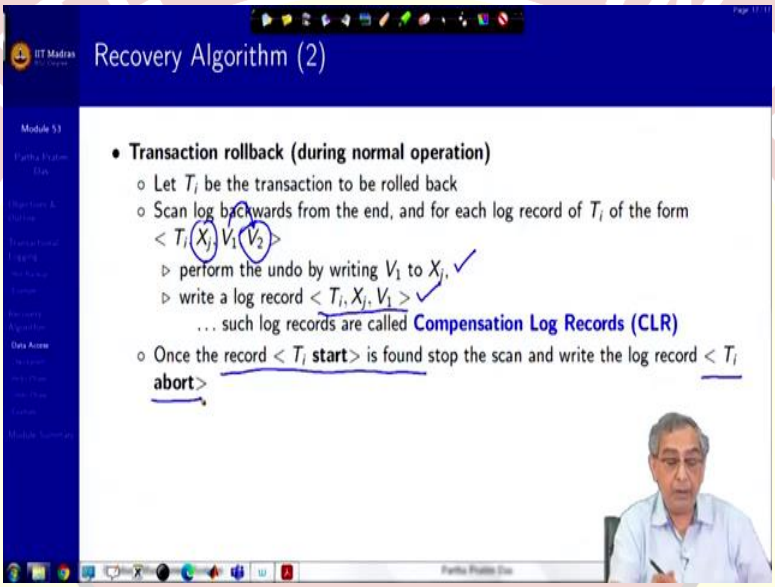


Now, what can happen otherwise? What can happen otherwise is the transaction rolls back but this rollback is during normal operation, that is transaction has been asked to roll back

because there was a deadlock. Say for some reason the transaction has to roll back. So, if Ti is a transaction to be rolled back, we discussed all this in the serial, I am just reminding you so that you can really you know get into it and understand it in the concurrent situation.

Then you, this is to be rolled back. So, what will you do? You will scan the log backwards from the end as much as has been done. So, at the point from where you are doing the rollback from there you keep going up and for each log record of update type you undo that. How do you undo?

(Refer Slide Time: 20:19)



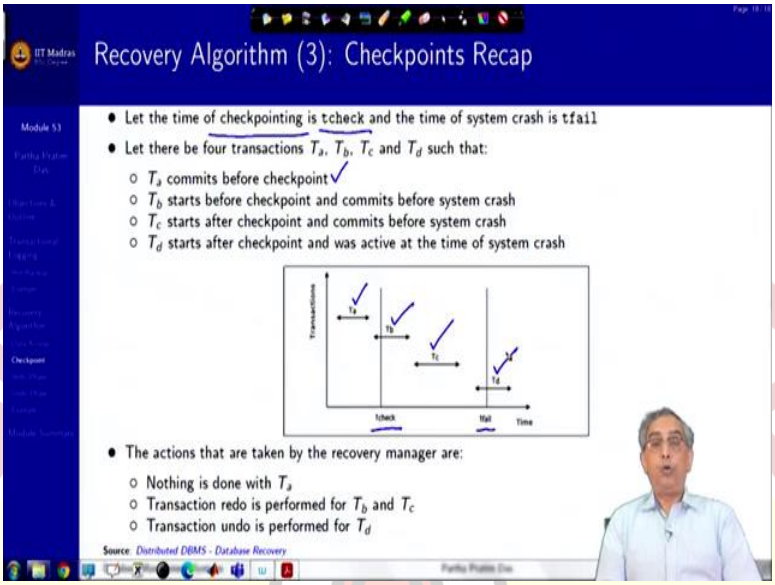
You undo because Xj was changed from V1 to V2 so the current value is V2. So, now you will change it back to V1. So, you will write V1to Xj and the fact that you have undone you write down in terms of, so you do not write a normal write log, what you write is a undo log which says that we had written change to V1 to V2 and now we have undone that change, V1 is back to so you do not write a second old value anymore because that certainly was the last disk that you had done on it.

And you write downs meaning that this is the compensation log, you have compensated for the earlier change that has been done. In this way going backwards certainly at the beginning of the transaction you had written Ti start so you will find that Ti start somewhere and when you find that you write down in the log Ti abort.

So, it shows, so if you now read the log you will find that Ti start a couple of update operations then a couple of CLR records, compensation log records from the point the transaction decided to roll back and then you have a Ti abort written in the log which shows

that you have aborted the operation and this transaction is no more having any effect. So, this is your basic recovery algorithm.

(Refer Slide Time: 21:51)



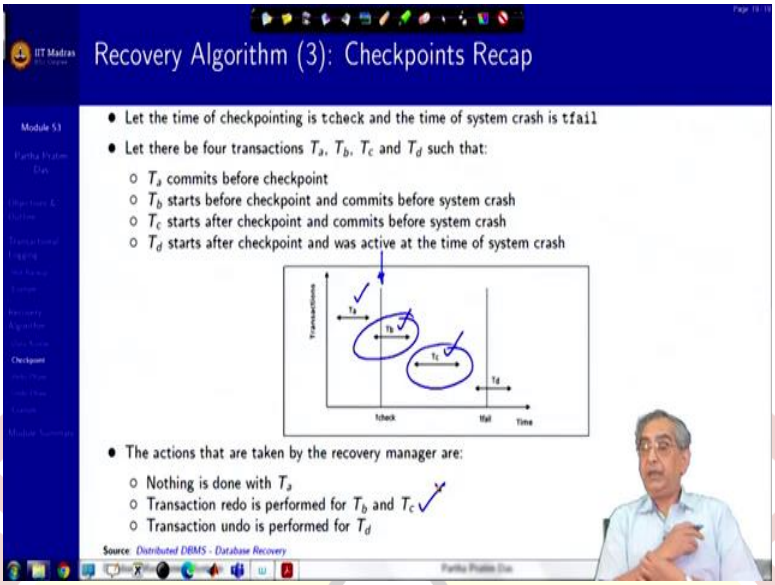
And then we saw that well if we just keep on doing this then your log will start becoming explosive, it will keep on accumulating item after item, after item and the basic assumption that it can have a hot backup which needs an expensive high speed nonvolatile memory will become a difficult proposition.

So, what you do at points you stop everything you say the world, the universe comes to a stop, you write back everything and put a clean slate on this. And that is what is known as the checkpoint. So, checkpoint let us say happens at this point t check when you are updating everything in the database.

So, there are four possibilities for the transactions with respect to the checkpoint and if in future a failure has happened. Now, you are not talking about normal roll back, you are saying if a failure has happened then what will happen? So, there are four types of transactions here. One, that is Ta which has committed before the checkpoint.

One, Tb which started before the checkpoint and committed after the checkpoint but before the failure. Third is Tc which started after the checkpoint and committed before the failure. And fourth is Td which started after the checkpoint before the failure but did not commit before the failure.

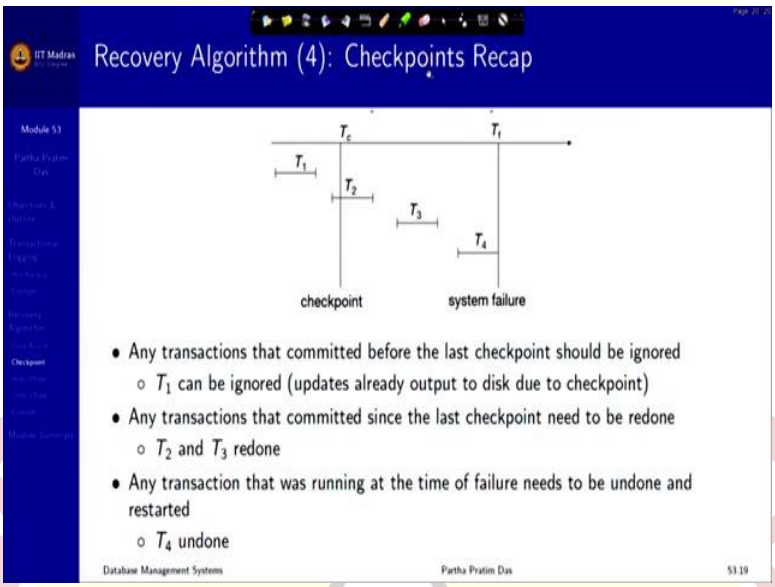
(Refer Slide Time: 23:30)



So, what we do need to do? Naturally for Ta you do not need to do anything, because you have done a check pointing here. So, all things that was done due to Ta are already updated. For Tb and Tc both of these had the changes happened after the check pointing, Tb was in transition, Tb was still executing when you did the checkpoint.

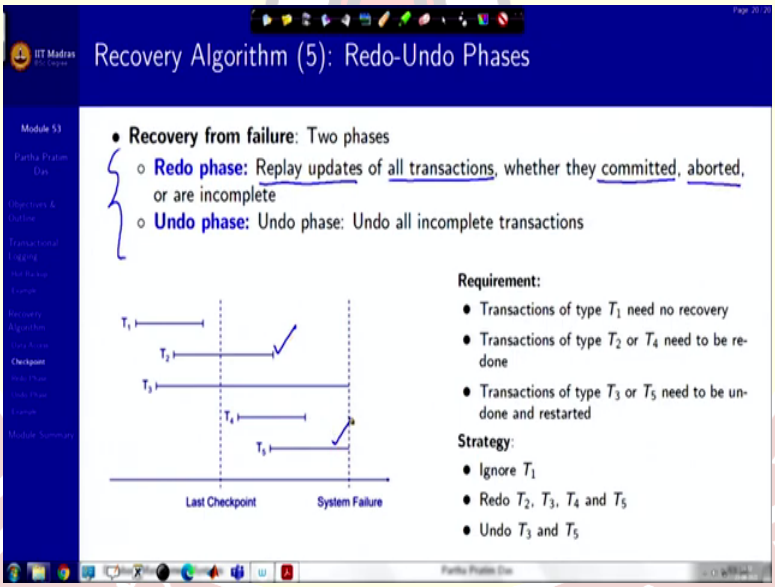
Tc happened all together after that but both of them have completed. So, they have to be to bring the system back to the present time when failure happened you will have to redo what Tb and Tc has done. So, you have to redo Tb and Tc. And once that has been done then you will have to do an undo for the transaction Td. So, this is basically the strategy that you will have to take.

(Refer Slide Time: 24:30)



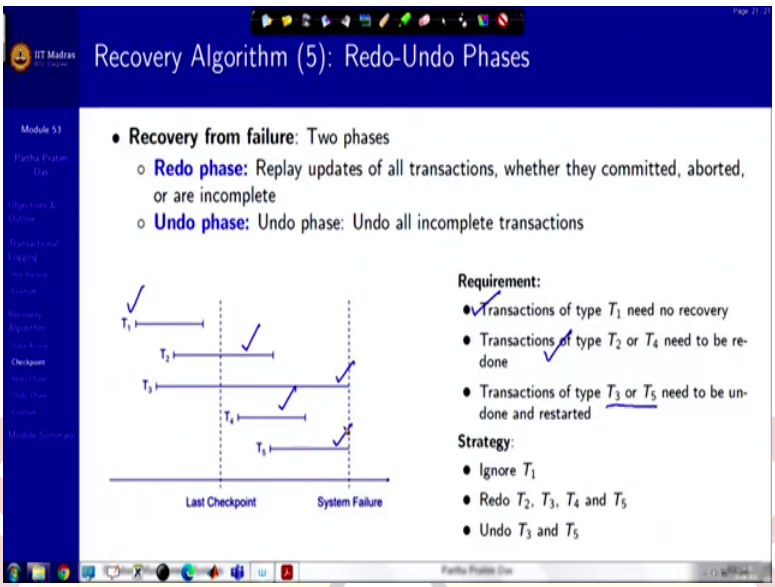
So, this is one example that we had seen earlier, let us see another example.

(Refer Slide Time: 24:38)



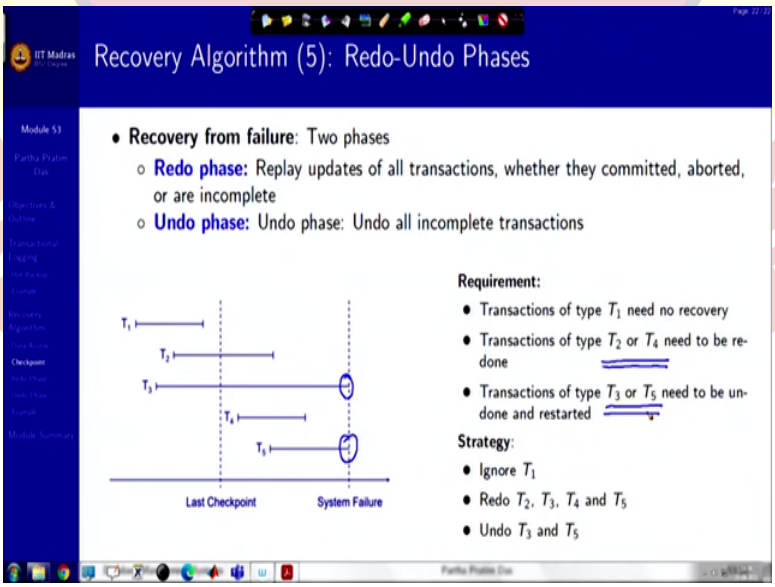
So, what you therefore have when you recover from a failure what you therefore have are two phases. One is a redo phase. So, in the redo phase you replay updates of all transactions, whether they are committed like this one, there may be something which is aborted or if they are incomplete like this one.

(Refer Slide Time: 25:11)



So, if you look into this example T1 everything was done before the check pointing, so there is no need for recovery. For transaction T2 and T4 you need to redo because they have done some changes which were committed but is getting lost, so you need, they need to be redone. Whereas transaction T3 and T5, T3 and T5, one started before the checkpoint therefore it and did not commit before the failure and the other started after the checkpoint but again did not commit before the failure, so they will have to be, you will have to undo their things.

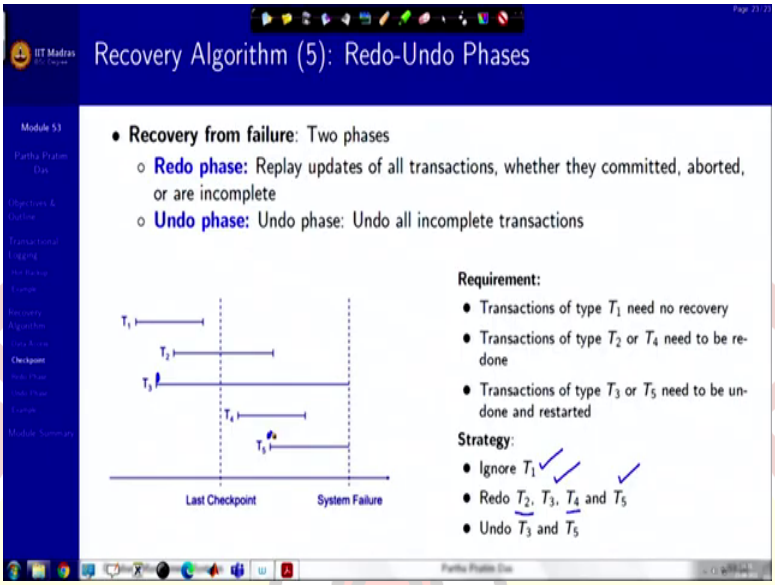
(Refer Slide Time: 26:05)



Now, the question is please try to understand this is a very tricky thing. For T3 and T5 the question is when the failure has happened their changes have also got lost. So, while the

requirement is you do the redo for T2, T4 logically and undo T3, T5 even the partial changes done by T3 and T5 are also lost because the system has failed.

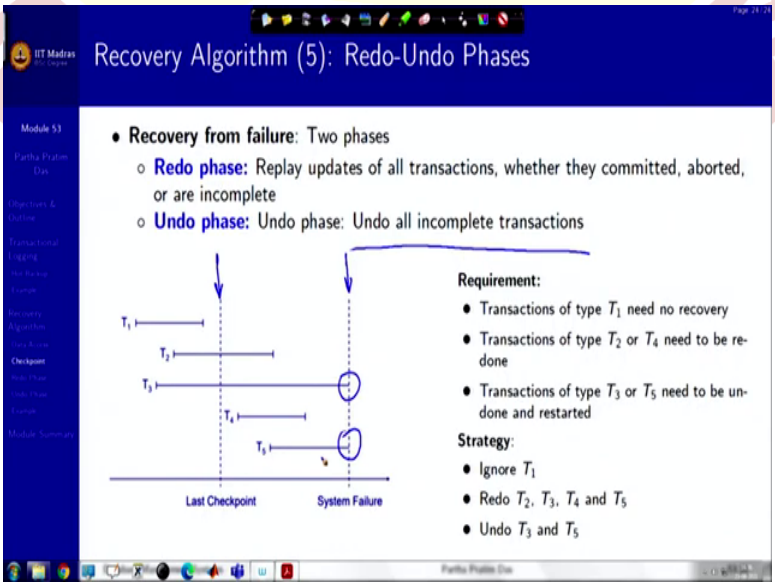
(Refer Slide Time: 26:41)



So, what you will have to do very interestingly you ignore T1, fine, but redo has to be done not only for T2, T4 also for T3, T5 which are partial, which are incomplete. So, that once you get to that incomplete state then you can undo them again to come to the point when they had actually started.

So, this of course would mean doing some unnecessary work because you are redoing for T3, T4, T3, T5 which will have to undo through T3, T5 again but keeping track of what changes are incompletely lost is not an easy task from the transaction log.

(Refer Slide Time: 27:32)

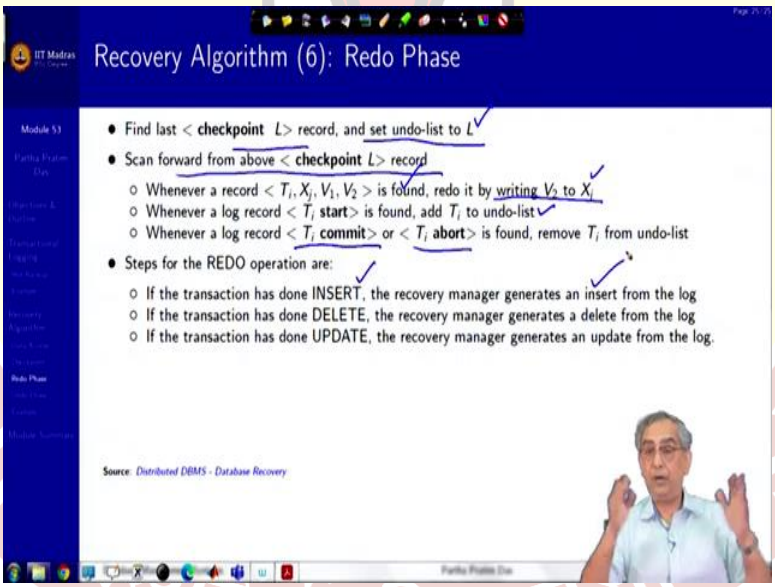


So, what you do very simply, so finally the rule of this redo undo phases turn out to be that for all transactions whether committed, aborted or incomplete things happening between the last checkpoint and the current system failure for all of them you do a redo. So, redo brings you to the point of system failure.

Then you, since you have lost what was happening in T3 because it is not just the data read write, T3 had its own local buffer, it had its own computations they have also got lost in the crash, so you cannot get those back.

So, for getting that you undo the transactions which are incomplete. For completed and aborted or aborted you do not need to do that, but for T3 and T5 you have to undo those changes so that you can get back to a consistent state which is not exactly the state when you had system failure because at that time T3 and T5 were already in execution but it gets you back to a consistent state of the database. So, this is the redo undo phasing and the basic algorithm.

(Refer Slide Time: 28:49)



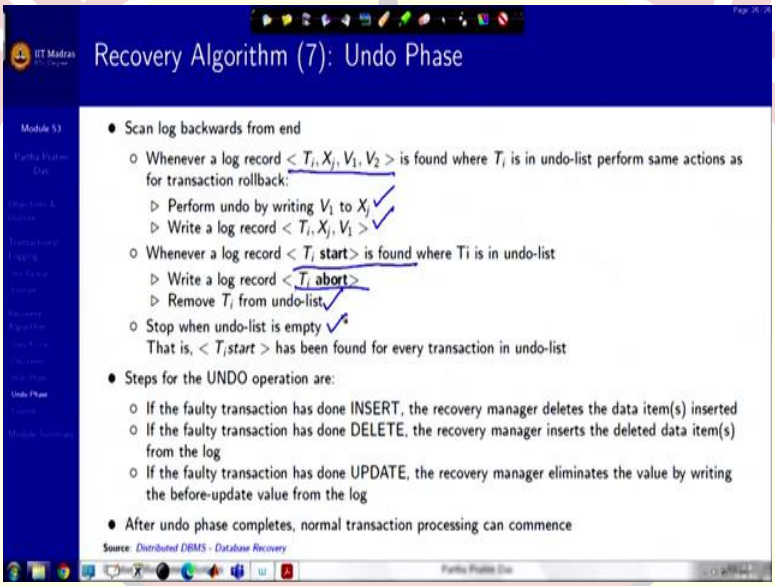
So, what will the redo phase do is very simple. Find the last checkpoint L, set undo list to L that is what you had at that check pointing. And scan forward from above the checkpoint list record and whenever you get this you redo because first you are redoing, so you again do that by writing V2 to X because you have to redo first.

So, you are starting from the top, coming down, redoing, if you find a start you put it to undo list because you might need to undo it. If you find a commit or an abort, then you remove it from the undo list that means it is a completed one. But if in the scan from the checkpoint

forward if you found a start but did not find a commit or abort that means that those are incomplete transactions.

So, starting from the undo list with the list of transactions live at the point of, at the checkpoint L you will add those transactions which have been further started and have not been committed or aborted. So, at the end of this process, at the end of the log what you get are a list of undo transactions which you will have to undo. Naturally for redo operation if there is a transaction has done an insert the recovery manager will do an insert from the log and so on so forth.

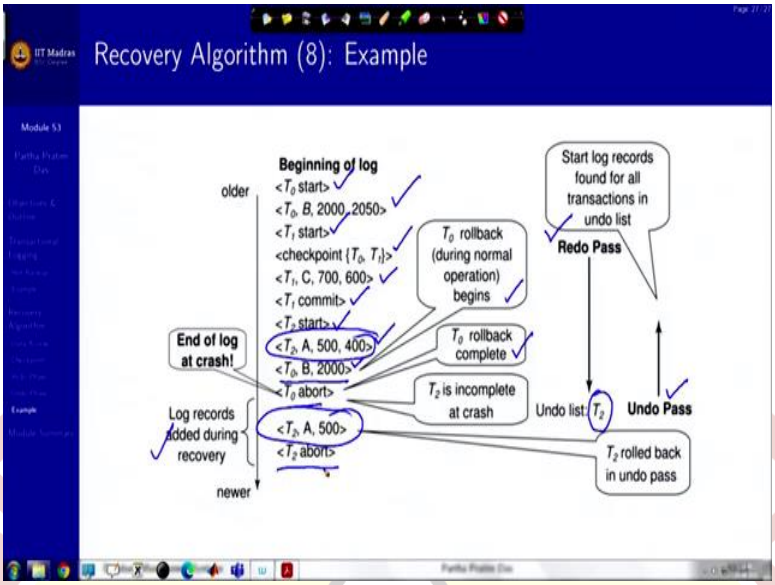
(Refer Slide Time: 30:29)



Naturally the undo phase is very similar to what we have discussed. Now, you look backwards whenever you find this you revert back the value, write a compensation log record whenever you find T start going backwards because now you are dealing only with the undo list which means only with the transactions which are incomplete. So, when you go back get the start you write the abort and then you remove it from the undo list, the undo for that transaction has been done.

Stop when this undo list becomes empty that is Ti start has been found for every transaction in the undo list. Naturally when you undo for an insert you will have to do a delete, for a delete you will have to do a insert, and naturally for update you will have to reverse. So, this is your basic redo undo phasing.

(Refer Slide Time: 31:26)

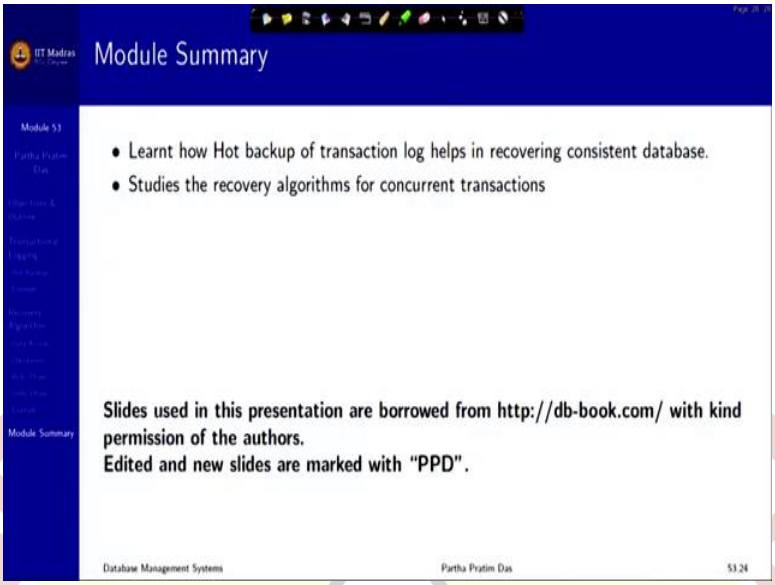


And if you just go through this example Ti starts it changes B from 2000 to 2050, T1 another transaction starts and you do a check pointing at that point. So, naturally your list contains T0, T1. Then after check pointing T1 changes C2 from 700 to 600, does a commit so it is a completed transaction.

T2 starts, the third transaction T2 starts, it makes changes to A. T0 decides to roll back so T0 rolls back writing the compensation log record 2000 and aborts, the normal operation. And T2 therefore at this point T1 has committed T0 has aborted so T2 at this point the crash happens, so at this point T2 is incomplete.

So, you have to do here with the redo pass when you first do the redo pass what will be left with at the end of redo is T2 and then you will have to do the undo pass with T2 which will have this so you have to write a compensation log record, this is what you are writing after you have after the crash when you are writing the recovery. And finally, you are done you write T2 abort there is nothing left in the undo list therefore you are done. So, this is how you can recover from a transaction using the redo and undo phase.

(Refer Slide Time: 33:14)



So, try to go through this carefully this is simple but the protocol is very, very important if to, it is again very important to understand that to be able to undo properly you have to redo the incomplete task in case of a system failure. This brings us to the end of this module. Thank you for your attention and see you in the next module where we will continue further.